Page No.:
Date: / /

Lecture - 13

Binary Search

Binary Search:

↳ Applied only on sorted array

Index	0	1	2	3	4
Array	2	7	11	13	18

Linear search:

```
for (int i=0;  i<n;  i++)
        if (arr[i]==target)
            cout<<" i";
```

O(N)

Binary search (Example):

0	1	2	3	4	5	6	7
2	4	7	18	23	29	32	38

Target = 32

Step 1: Find mid:

(1) * $mid = \frac{low + high}{2}$

low = 0 , high = 7

$mid = \frac{0+7}{2} = 3$

Teacher's Signature.....

Check (arr[mid] == target)

arr[3] == 32

18 == 32 (False)

18 < 32

arr[mid] < target

⇓

Start = mid + 1 = 4

end = 7

(2) * Mid = $\frac{4+7}{2}$ = $\frac{11}{2}$ = 5

arr[5] = target

arr[5] == 32

29 == 32 (False)

29 < 32

arr[mid] < target

Start = mid + 1 = 6

end = 7

(3) * Mid = $\frac{6+7}{2}$ = 6

arr[6] == target

32 == 32 (True)

return 6

Page No.:
Date: / /

Algorithm :

① Find mid element

mid = (low + high) / 2;

② Check arr[mid] is equal to target

case 1: arr[mid] = target

return mid

case 2: target > arr[mid]

search in right

low = mid+1

case 3: target < arr[mid]

search in left

high = mid - 1;

Code :

```
int mid;
int low = 0;
int high = n-1;

while ( low <= high) {
    mid = (low + high) / 2;

    if ( arr[mid] == target) {
        cout << mid;
        return;
    }
```

Teacher's Signature.....

```
        else if (target > arr[mid]) {
                low = mid + 1;
            }
        else {
                high = mid - 1;
            }
    }
```

Time Complexity:

Worst case : O(log N).

* Left most and right most index :

Given : Sorted array
↳ With duplicate element.

Task : find first & last occurence of element x.

If element is not present return {-1, -1} as pair.

Example:

N = 9

V[] : 1 3 5 5 5 5 67 123 125

X = 5

output = 2 5

1	3	5	5	5	5	67	123	125

For finding leftmost occurence of x,

0	1	2	3	4	5	6	7	8
1	3	5	5	5	5	67	123	125

$$mid = \frac{0+8}{2} = 4$$

arr[mid] == target (True)

leftMost = ~~-1~~ 2 end = mid - 1 :

0	1	2	3
1	3	5	5

$$mid = \frac{0+3}{2} = 1$$

arr[mid] == target (False)

arr[mid] < target

↳ start = mid + 1

2	3
5	5

mid = 2

arr[mid] == target (true)

end = mid - 1

↳ No Exist.

leftMost = 2

Teacher's Signature.....

For rightMost :

0	1	2	3	4	5	6	7	8
1	3	5	5	5	5	67	123	125

rightMost = -1 ~~4~~ 5 mid = 4

arr[mid] == target (true)

start = mid + 1

5	6	7	8
5	67	123	125

mid = 6

arr[mid] = target (false)

arr[mid] > target

end = mid - 1

5

5

mid = 5

arr[mid] = target (true)

start = mid + 1

↳ Not Exist.

rightMost = 5

return {2, 5}

code :

```
pair<long, long> indexes(vector<long, long> v, long long x){
        long long start = 0;
        long long end = v.size() - 1;
        long long leftindex = -1;
        long long rightindex = -1;
        long long mid;
```

Teacher's Signature.....

```
While ( start <= end ) {
    mid = end + (start - end) / 2;
        if ( v[mid] == x ){
            leftindex = mid;
                end = mid - 1;
        }
        else if ( v[mid] > x ){
                end = mid - 1;
        } else {
                start = mid + 1;
        }
}
start = 0;
end = v.size() - 1;
 While ( start <= end ) {
        mid = end + (start - end) / 2;
        if ( v[mid] == x ){
            rightindex = mid;
            start = mid + 1;
        }
        else if ( v[mid] > x ){
                end = mid - 1;
        }
        else {
                start = mid + 1;
        }
    }
    return { leftmost leftindex, rightindex };
}
};
```

Teacher's Signature.....

Page No.:
Date: / /

* search insert position of K in a sorted array:

num = {1, 3, 5, 6} target = 5

output = 2

case 1: target present in array

↳ then return index.

case 2: target does not present in array

num = {1, 3, 5, 6} target = 2

output = 1

For this we apply Binary search :-

1 (3) 5 6

Mid = $\frac{0+3}{2}$ = 1

2 < 3 index = 1

{ , }

$\frac{0+0}{2}$ = 0

index = 0 + 1 = 1

1 < 2

→ अगर target < arr[mid]

↳ index = mid

→ अगर target > arr[mid]

↳ index =

Teacher's Signature.....

Page No.
Date / /

Code:

```
int searchInsertK(vector<int> arr, int N, int target){
        int start = 0;
        int end = N-1;
        int index;

    while (start <= end) {
        int mid = start + (start - end) / 2;
        if (arr[mid] == target) {
            index = mid;
            break;
        }
        else if (arr[mid] < target) {
            start = mid + 1;
            index = mid + 1;
        }
        else {
            end = mid - 1;
            index = mid;
        }
    }
    return index;
}
};
```

Teacher's Signature.....

Page No.:
Date: / /

* Peak Element :

Given : Array arr[]
Size N

Task : Return index of any one of its peak element.

peak element is an element whose value is not smaller than the adjacent element.

Example :

N = 3
arr[] = {1, 2, 3}

↓

{1, 2, 3}

1 > 2 (false) Not peak

↓

{1, 2, 3}

2 > 1 (True) } False Not peak
2 > 3 (False }

↓

{1, 2, 3}

3 > 2 (True)
3 last element

peak element.

Teacher's Signature.....

→ arr[i-1] < arr[i] > arr[i+1]

Code:

```
int peakElement (int arr[], int n) {
        int start = 0;
        int end = n-1;

        while (start <= end) {
            int mid = end + (start - end) / 2;

        if (arr[mid] > arr[mid + 1] && arr[mid] >
                    arr[mid - 1])
            return mid;

        else if (arr[mid] > arr[mid - 1] &&
                    arr[mid] < arr[mid + 1])
            start = mid + 1;

        else
            end = mid + 1;
        }
        return -1;
    }
};
```

Teacher's Signature.....